

Roteiro de Apresentação — Cache Aside e Performance de APIs

Arquitetura de Software — 5º período | PUC Minas

Como usar este roteiro: Cada seção cobre um slide. O bloco "*O que dizer*" traz o texto de apoio para a fala. O bloco "*Termos-chave*" explica os conceitos para que ninguém fique perdido durante perguntas da banca.

Slide de Capa

O que dizer:

"Olá, somos do grupo X. Hoje vamos apresentar nosso trabalho de Arquitetura de Software, que implementa e mede o padrão Cache Aside em uma API REST. O nosso objetivo foi provar, com números reais, que uma camada de cache bem implementada reduz latência, alivia o banco de dados e aumenta o número de requisições que o sistema aguenta."

Termos-chave:

- **Cache Aside** — padrão arquitetural em que a aplicação gerencia o cache manualmente (não o banco, não o framework)
 - **API REST** — interface de programação que segue o estilo arquitetural REST; recebe requisições HTTP e retorna dados (geralmente JSON)
 - **Stack tecnológica** — conjunto de tecnologias usadas: Rails (servidor), PostgreSQL (banco relacional), Redis (cache em memória), Docker Compose (orquestração), k6 (teste de carga), Grafana (visualização)
-

Slide 1 — Problema

O que dizer:

"O problema que motivou este trabalho é clássico em sistemas reais. Imagine uma loja online: cada vez que um usuário abre a listagem de produtos, o servidor vai ao banco de dados, executa uma query SQL, traz os dados e devolve. Se 200 usuários abrirem a mesma página ao mesmo tempo, o banco recebe 200 queries idênticas. Como os dados de catálogo raramente mudam — um produto novo é criado uma vez, mas lido milhares de vezes — esse comportamento é um desperdício. O banco fica sobrecarregado, a latência aumenta e o sistema começa a engolir menos requisições por segundo."

Termos-chave:

- **PostgreSQL** — banco de dados relacional open-source; aqui guarda clientes, produtos e pedidos em tabelas
- **Gargalo (bottleneck)** — ponto do sistema que limita a performance do todo; quando o banco é lento, o servidor inteiro fica lento esperando por ele
- **Latência** — tempo que passa desde o momento em que uma requisição chega até a resposta sair; medida em milissegundos (ms)

- **Throughput** — quantidade de requisições processadas por segundo (req/s); mede capacidade total do sistema
 - **Dados de catálogo** — dados que mudam com pouca frequência mas são lidos constantemente: lista de produtos, detalhes de um produto, listagem de clientes
-

Slide 2 — O Padrão Cache Aside

O que dizer:

"A solução é colocar uma camada de memória entre a aplicação e o banco. No padrão Cache Aside, o fluxo de leitura funciona assim: a aplicação pergunta primeiro ao Redis. Se a resposta já estiver lá — chamamos isso de HIT — ela retorna imediatamente, sem tocar o banco. Se não estiver — MISS — a aplicação consulta o banco, guarda o resultado no Redis e devolve ao cliente. Na próxima vez que alguém pedir o mesmo dado, será um HIT. Para escritas, a estratégia é diferente: persistimos no banco e apagamos a chave do Redis. Não atualizamos o cache — apagamos. Na próxima leitura, o cache será reconstruído com dado fresco."

Termos-chave:

- **Redis** — banco de dados em memória (RAM), extremamente rápido; usado aqui como cache. Uma leitura no Redis leva ~0,1 ms; no PostgreSQL, em carga, pode levar 10–20 ms ou mais
 - **Cache HIT** — o dado pedido já estava no Redis; resposta imediata, banco não é consultado
 - **Cache MISS** — o dado não estava no Redis; precisa ir ao banco, depois grava no Redis para as próximas requisições
 - **Invalidar** — apagar uma chave do Redis quando o dado no banco muda, forçando que a próxima leitura busque o dado atualizado
 - **Write-through** — estratégia alternativa onde a escrita atualiza banco e cache ao mesmo tempo; mais complexo de garantir atomicidade; optamos por invalidar porque é mais simples e mais seguro
 - **Atomicidade** — garantia de que duas operações (gravar no banco e gravar no cache) acontecem juntas ou nenhuma delas acontece; difícil de garantir sem transações distribuídas
-

Slide 3 — Arquitetura da Solução

O que dizer:

"Aqui está a arquitetura completa. O cliente HTTP — pode ser um browser, um app mobile, o k6 — faz uma requisição HTTP ao nosso servidor Rails. O Rails não vai direto ao banco: ele passa pelo CacheService. O CacheService.fetch verifica o Redis. Se for HIT, devolve. Se for MISS, consulta o PostgreSQL, grava no Redis e devolve. Para escritas, o CacheService.invalidate apaga as chaves relevantes. Além disso, mantemos contadores de estatísticas no próprio Redis através do CacheStats, acessíveis em GET /stats. Para observabilidade, os logs do Rails são capturados pelo Promtail, enviados ao Loki, e visualizados no Grafana."

Termos-chave:

- **Rails 7.2 (API-only)** — framework web em Ruby configurado apenas para APIs; não serve HTML, só JSON

- **CacheService** — módulo Ruby que criamos para centralizar toda a lógica de cache; todos os controllers chamam apenas este módulo, nunca o Redis diretamente
 - **CacheService.fetch(key) { bloco }** — método principal; recebe uma chave (ex: "products:all") e um bloco de código (a consulta ao banco). Se a chave existir no Redis, retorna o valor cacheado; senão, executa o bloco, armazena o resultado e o retorna
 - **CacheService.invalidate(*keys)** — apaga uma ou mais chaves do Redis de uma vez
 - **CacheStats** — contador armazenado no Redis (em um hash) que registra hits, misses e invalidações; exposto via endpoint `/stats`
 - **CACHE_ENABLED=true|false** — variável de ambiente que liga/desliga o cache sem mudar o código; essencial para rodar os dois cenários do benchmark com a mesma imagem Docker
 - **Promtail → Loki → Grafana** — pipeline de observabilidade: Promtail lê os arquivos de log do Rails, envia para o Loki (banco de dados de logs), e o Grafana exibe dashboards com gráficos de HITs, MISSes e latência
 - **k6** — ferramenta de teste de carga; simula múltiplos usuários fazendo requisições simultâneas
-

Slide 4 — Metodologia dos Testes

O que dizer:

"Para medir o impacto real do cache, precisávamos de um teste controlado. Usamos o k6 com 200 usuários virtuais simultâneos, a uma taxa de 40 requisições por segundo, por cerca de 220 segundos. Cada usuário simulado segue uma jornada que representa um uso realista do sistema: lista produtos, detalha um produto, consulta pedidos, e 25% das vezes faz uma compra — que é exatamente a operação que invalida o cache. Rodamos o mesmo script duas vezes: uma com cache desligado (**CACHE_ENABLED=false**) e outra com cache ligado (**CACHE_ENABLED=true**), sempre com o banco com os mesmos dados e o Redis zerado antes de cada run."

Termos-chave:

- **k6** — ferramenta open-source da Grafana Labs para testes de carga; o script descreve o comportamento de um usuário virtual e o k6 executa em paralelo com centenas deles
 - **Usuários virtuais (VUs)** — threads/coroutines que o k6 cria para simular usuários simultâneos; 200 VUs significa 200 jornadas acontecendo ao mesmo tempo
 - **Ramp-up progressivo** — os 200 usuários não entram todos de uma vez; aumentam gradualmente para simular tráfego realista e evitar pico artificial
 - **Taxa de chegada (40 req/s)** — ritmo de novas requisições por segundo; independente do tempo de resposta — mesmo que o servidor esteja lento, novas requests continuam chegando nessa taxa
 - **GET /products** — busca a lista completa de produtos; operação de leitura pesada, candidata ideal ao cache
 - **GET /orders** — busca pedidos com JOIN nas tabelas de itens; a query mais custosa no banco
 - **POST /orders (25%)** — criação de pedido; representa a escrita que invalida o cache; apenas 25% das iterações para simular o mundo real onde leituras são muito mais frequentes
 - **Seed** — dados iniciais inseridos no banco antes dos testes; 300 clientes, 500 produtos, 2.000 pedidos; idênticos nos dois cenários para comparação justa
 - **FLUSHALL** — comando Redis que apaga tudo do cache; executado antes de cada run para garantir que o cenário **cache-on** começa do zero, sem vantagem de aquecimento
-

Slide 5 — Resultados: Latência

O que dizer:

"Os resultados de latência mostram uma melhora consistente de aproximadamente 55% em todos os percentis. A latência média caiu de 12,3 ms para 5,5 ms. O p95 — que é o percentil mais usado para definir SLAs — caiu de 17,3 ms para 7,7 ms. Isso significa que 95% das requisições foram respondidas em menos de 7,7 ms com cache ligado. O ganho é uniforme porque a maioria das leituras é atendida pelo Redis, que é muito mais rápido que o banco sob carga."

Termos-chave:

- **Latência média** — média aritmética do tempo de resposta de todas as requisições; boa para visão geral, mas pode ser distorcida por outliers
- **Percentil p90** — 90% das requisições foram respondidas em menos que este valor; os 10% mais lentos ficaram acima
- **Percentil p95** — 95% das requisições foram abaixo; o valor mais usado em contratos de SLA (ex: "garantimos que 95% das requests respondem em menos de X ms")
- **Percentil p99** — 99% das requisições abaixo; mede o "cauda longa" — os casos extremamente lentos
- **Latência máxima** — a requisição mais lenta de todo o teste; útil para identificar casos de timeout
- **SLA (Service Level Agreement)** — acordo de nível de serviço; define o tempo máximo aceitável de resposta; o p95 é o percentil padrão da indústria para esse tipo de contrato
- **Por que todos os percentis melhoraram ~55%?** — porque o cache intercepta a maioria das leituras antes de chegarem ao banco; o banco só é consultado nos MISSES (~40% das requisições), então a carga média por requisição cai drasticamente

Slide 6 — Resultados: Throughput

O que dizer:

"O throughput mais que dobrou. Sem cache, o servidor entregou 2.982 requisições totais a 13,5 req/s. Com cache, foram 6.044 requisições a 28 req/s — um aumento de 107%. As iterações completas — que representam jornadas de cliente inteiras — subiram de 728 para 1.617, ou seja, 2,2 vezes mais clientes atendidos no mesmo período. As iterações dropadas também caíram: com o servidor mais rápido, menos jornadas foram abortadas por timeout."

Termos-chave:

- **Throughput (req/s)** — requisições por segundo processadas com sucesso; a principal métrica de capacidade do sistema
- **Iterações concluídas** — o k6 chama de "iteration" cada execução completa da função do usuário virtual (todas as 4 etapas: catálogo → produto → pedidos → compra); representa uma jornada de cliente do início ao fim
- **Iterações dropadas** — jornadas que foram abortadas antes de completar, geralmente por timeout; quando o servidor está sobrecarregado, o k6 descarta iterações em vez de deixá-las acumular indefinidamente
- **Por que o throughput dobrou?** — quando o cache responde em ~1 ms em vez de ~15 ms, o servidor fica livre mais rápido para atender a próxima requisição; a mesma capacidade de hardware entrega

mais trabalho por segundo

- **Gargalo resolvido** — o banco era o limitante; com o cache absorvendo 60% das leituras, o banco recebeu 43,5% menos queries, liberando CPU e I/O para as queries que de fato precisam ir até ele

Slide 7 — Cache Hit, Miss e Invalidação

O que dizer:

"Durante o run com cache, registramos 420 HITs, 291 MISSES e 49 invalidações. Isso dá uma taxa de acerto de 59,1%. Para cada 10 requisições de leitura, quase 6 foram respondidas pelo Redis sem tocar o banco. As 49 invalidações correspondem exatamente às compras realizadas durante o teste — cada POST /orders apaga as chaves `orders:all` e `orders:<id>` do Redis. O efeito no banco é direto: sem cache, 1.422 queries; com cache, 804 — uma redução de 43,5%."

Termos-chave:

- **Hit Rate (taxa de acerto)** — proporção de leituras respondidas pelo cache sem ir ao banco; $\text{hits} / (\text{hits} + \text{misses}) \times 100$; 59,1% significa que 59 em cada 100 leituras foram servidas pelo Redis
- **Por que o hit rate não é 100%?** — os primeiros MISSES de cada chave são inevitáveis (cold start); escritas geram invalidações que forçam novos MISSES; queries com parâmetros diferentes geram chaves diferentes
- **Invalidações (49)** — cada compra (POST /orders) chama `CacheService.invalidate("orders:all", "orders:<id>")`; as 49 invalidações batem exatamente com o volume de compras (25% de 200 iterações × 1 compra)
- **Consultas ao banco** — contadas pelo CacheStats; sem cache = toda leitura é uma query; com cache = só MISSES e escritas viram queries. Redução de 43,5% direto da taxa de 59% de HITs
- **orders:all vs orders:<id>** — duas chaves diferentes; uma guarda a lista completa de pedidos, outra guarda um pedido específico; ao criar um pedido novo, ambas precisam ser invalidadas porque a lista mudou

Slide 8 — Trade-offs Arquiteturais

O que dizer:

"Nenhuma arquitetura é perfeita — cache-aside resolve problemas e cria outros. O ganho de latência e throughput é claro. A consistência é o preço: sem cache, o banco é sempre a fonte de verdade e a leitura reflete o estado atual. Com cache, existe uma janela de tempo em que o Redis pode ter um dado levemente desatualizado. Nossa estratégia `invalidate-on-write` minimiza isso: a chave é apagada imediatamente após a escrita, então a janela de inconsistência dura apenas o tempo entre a escrita e a próxima leitura reconstruir o cache — na prática, milissegundos. O TTL de 300 segundos é uma rede de segurança: se uma invalidação falhar por alguma razão, em 5 minutos a chave expira sozinha e o sistema volta ao estado correto."

Termos-chave:

- **Consistência forte** — garantia de que toda leitura retorna o dado mais recente, sem exceção; o banco relacional oferece isso; o cache não garante

- **Consistência eventual** — o sistema garante que, em algum momento, todos os nós convergem para o mesmo valor; a leitura pode retornar dado levemente desatualizado durante a janela de transição
- **invalidate-on-write** — estratégia de invalidação: ao escrever, apaga a chave; a próxima leitura vai ao banco e reconstrói o cache com dado fresco. Alternativa seria **update-on-write** (atualizar o cache junto), mas exigiria garantias de atomicidade
- **TTL (Time-To-Live)** — tempo máximo de vida de uma chave no Redis; configurado como 300 segundos (5 minutos); quando o tempo expira, o Redis apaga a chave automaticamente; funciona como "invalidação automática de emergência"
- **Fallback automático** — se o Redis ficar offline, o `CacheService.fetch` verifica `enabled?`; a flag `CACHE_ENABLED` foi implementada para que o sistema degrade graciosamente para o banco, sem crash
- **Condição de corrida (race condition)** — dois usuários fazem requisição simultaneamente no momento de MISS; ambos consultam o banco e gravam a mesma chave no Redis; inofensivo porque os dados são idênticos (mesma query, mesmo resultado)
- **Complexidade adicional** — com cache, o time precisa gerenciar: quais chaves invalidar em cada operação de escrita, qual TTL usar, o que fazer quando o Redis fica indisponível; isso não existia sem cache

Slide 9 — Conclusão

O que dizer:

"Resumindo: implementamos o padrão Cache Aside em uma API Rails com Redis, medimos o impacto com o k6 e todos os objetivos do enunciado foram alcançados. A latência caiu 55%, as consultas ao banco diminuíram 43,5%, o throughput mais que dobrou e demonstramos o ciclo completo de HIT, MISS e invalidação com números reais. As lições mais importantes são: primeiro, cache-aside dá controle total à aplicação — e também toda a responsabilidade de invalidação correta; segundo, invalidar é mais seguro do que atualizar; terceiro, o TTL não é otimização de memória — é uma rede de segurança essencial; e quarto, 59% de hit rate já entrega ganho expressivo. Sistemas de produção com dados mais estáveis atingem facilmente 80 a 95%."

Termos-chave:

- **Hit rate 80–95% em produção** — sistemas reais com catálogos estáveis (e-commerce, conteúdo editorial) têm hit rates muito maiores porque os dados mudam raramente; nossos 59% refletem o volume alto de escritas (25% das iterações são compras) e o período curto de teste
- **Lição 1: controle e responsabilidade** — diferente de soluções automáticas (como caching de ORM), o cache-aside exige que o desenvolvedor saiba exatamente quais chaves invalidar em cada escrita; mais código, mais poder
- **Lição 2: invalidar > atualizar** — se você atualiza o cache na escrita e a operação falha no meio, pode ficar com banco e cache em estados diferentes; apagando, você garante que a próxima leitura sempre vai ao banco para reconstruir
- **Lição 3: TTL como rede de segurança** — não é só "para liberar memória"; é a garantia de que, mesmo que a invalidação falhe por bug ou falha de rede, o dado desatualizado vai expirar e o sistema se autocorrige

Dicas para a Apresentação

Se perguntarem sobre consistência:

"Nossa estratégia é consistência eventual com janela mínima. A chave é apagada imediatamente após a escrita. Não existe dado desatualizado *ativo* no Redis — o dado foi removido. A única janela de inconsistência seria se dois eventos ocorressem em milissegundos: alguém ler durante o MISS antes da invalidação, e a invalidação falhar. Para dados de catálogo, esse risco é aceitável."

Se perguntarem "por que não usar write-through?":

"Write-through exige que a escrita no banco e no cache aconteçam atomicamente. Sem transação distribuída, se o banco gravar e o Redis falhar, ficamos com inconsistência. Invalidar é mais simples: apaguei a chave, pronto. A próxima leitura reconstrói do zero."

Se perguntarem "por que Redis e não Memcached?":

"Redis suporta estruturas de dados mais ricas (hashes, listas, sets), tem persistência opcional, e o Rails tem suporte nativo via `redis-rails`. Para os nossos CacheStats (contadores), usamos hashes do Redis — não seria possível com Memcached facilmente."

Se perguntarem sobre o hit rate de 59%:

"É esperado neste contexto de teste. Temos 200 usuários simultâneos, 25% de escritas (que invalidam), e o teste começa com cache frio. Em produção, com tráfego mais uniforme e menos escritas, o hit rate seria bem maior — facilmente 80%+."

Se perguntarem sobre o custo do Redis:

"Redis roda em memória RAM. Para os dados deste projeto (300 clientes, 500 produtos, 2.000 pedidos em JSON), o consumo é de poucos megabytes — negligível. O ganho de 2× throughput e 55% menos latência compensa amplamente o custo de uma instância Redis."

Divisão sugerida de falas por integrante

Slides	Conteúdo	Integrante sugerido
Capa + Slide 1	Problema e contexto	Integrante A
Slides 2 + 3	Padrão e arquitetura	Integrante B
Slides 4	Metodologia dos testes	Integrante C
Slides 5 + 6	Resultados latência e throughput	Integrante D
Slides 7 + 8	Hit/Miss, trade-offs	Integrante A
Slide 9	Conclusão + perguntas	Todos